

Digit Recognition

Building a CNN-based classifier for digits 0–9
from noisy, multi-speaker audio — trained from scratch

Submitted by: Group_52
Aryan Mittal(230218)

EE708 Course Project

10 Digit Classes

CNN Architecture

**From
Scratch** No Pretrained
Weights

**Mel
Spectro.** Input Features

What Are We Solving?



Multi-Speaker

Test audio contains unseen speakers never seen during training. Models must learn digit patterns, not speaker identity.



Noisy Audio

Recordings include background noise, augmentation, and signal distortions — simulating real-world conditions.



No Pretraining

No Wav2Vec, HuBERT, or Whisper allowed. Model must be built and trained from scratch using only provided data.

Goal: Classify spoken digits (0–9) from .wav clips = generalizing across speakers, noise, and distortions

Understanding the Data

train.csv: Labeled training audio with id → label mapping

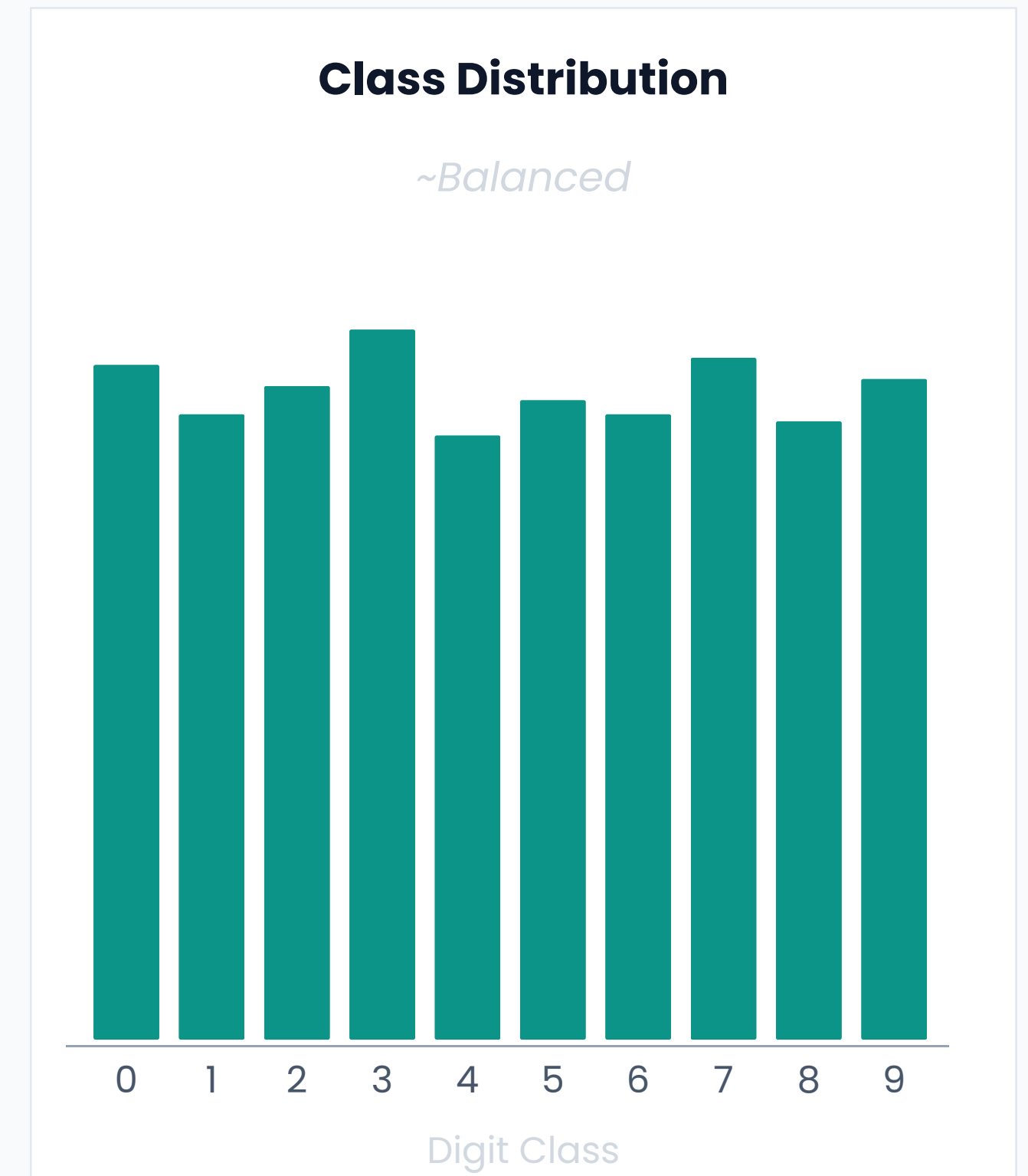
test.csv: Unlabeled test set for submission generation

Classes: 10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

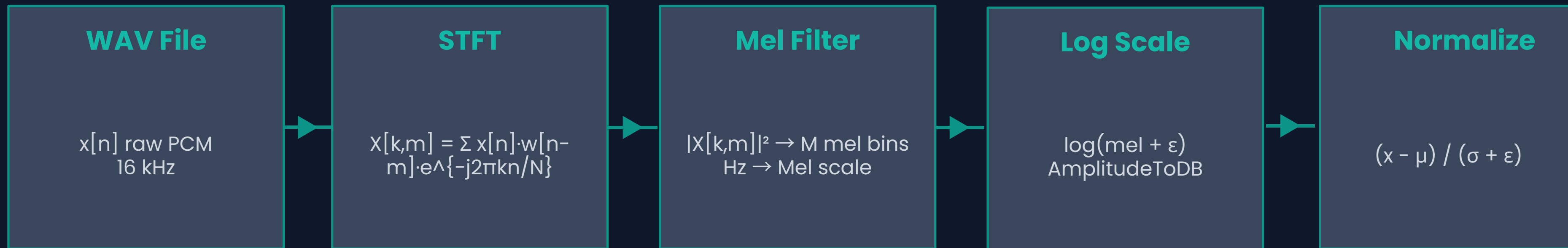
Format: .wav files, resampled to 16 kHz

Duration: ~1 second clips (variable — padded/trimmed)

Speakers: Multiple speakers; test set has unseen speakers



From Raw Audio to Mel Spectrogram



Key Math

Mel scale: **$m = 2595 \cdot \log_{10}(1 + f/700)$**

Why?

Human hearing is logarithmic in frequency.

STFT window $N=1024$, hop=128 $\rightarrow$ fine time resolution

128 mel bins $\rightarrow$ spectrogram shape: (1, 128, 256)

Why Mel Spectrogram?

- ✓ Captures frequency $\times$ time jointly
- ✓ Mimics human auditory perception
- ✓ 2D image $\rightarrow$ CNN can process it
- ✓ More robust than raw MFCC for digit tasks

Final Input Tensor: (1, 128, 256) — 1 channel, 128 freq bins, 256 time steps

Preventing Overfitting via Augmentation

SpecAugment

Mask F freq-bins & T time-steps with 0

2× freq masks (max 15 bins)
2× time masks (max 40 steps)
Forces model to not over-rely on any single freq/time region

Gaussian Noise

$$\tilde{x} = x + \epsilon, \epsilon \sim N(0, 0.05^2)$$

Adds small random perturbations to mel values
Simulates microphone noise & recording artifacts

Random Volume

$$\tilde{x} = \alpha \cdot x, \alpha \sim U(0.7, 1.3)$$

Scales amplitude uniformly
Simulates different recording loudness levels

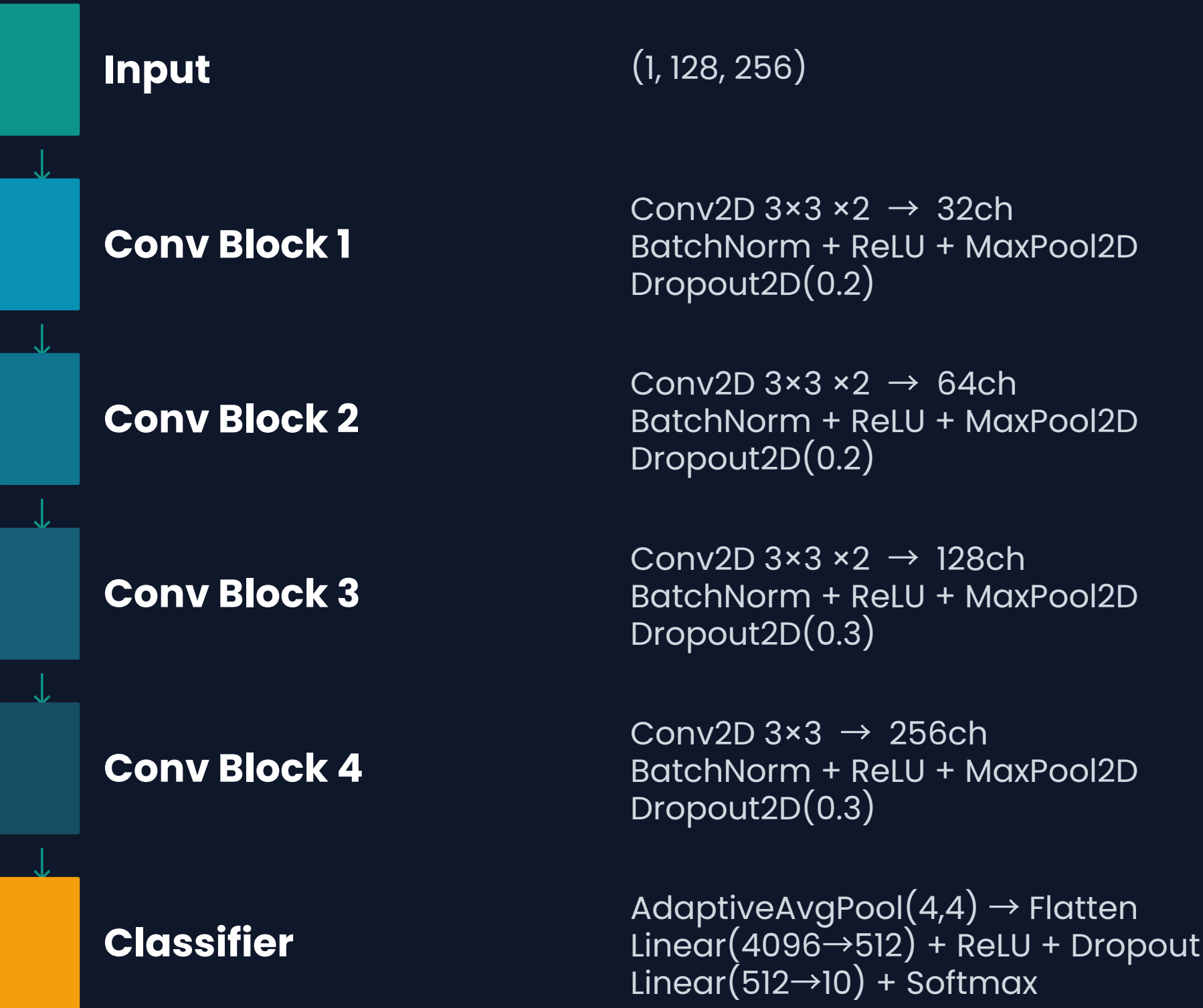
Time Shift

$$\tilde{x}[t] = x[(t - \Delta) \bmod T]$$

Circular roll by $\Delta \sim U(-20, 20)$ steps
Simulates speech onset timing variation

- Augmentation applied ONLY during training — val/test datasets use clean mel spectrograms (no aug leak)

4-Block CNN — Trained From Scratch



Design Choices

BatchNorm

Stabilizes gradients;
allows higher LR

MaxPool2D

Halves spatial dims;
builds hierarchy

Dropout2D

Drops entire feature
maps — strong reg.
Randomly drop feature maps.

AdaptiveAvg Pool

Fixed output size
regardless of input

CrossEntropy Loss

$\log p(y|x)$ — ideal
for classification

Optimization & Training Details

Optimizer Adam	$lr = 1 \times 10^{-3}$, $weight_decay = 1 \times 10^{-4}$ Momentum: $\beta_1=0.9$, $\beta_2=0.999$	Train / Val Split 90% — Training Set (augmented) 10% — Validation Set (clean) Key: • Stratified split — all 10 classes present in both train & val • Separate Dataset objects — val set never sees augmentation • Best model saved by val accuracy
Scheduler Cosine Annealing	$T_max = 30$ epochs $lr_t = lr_min + \frac{1}{2}(lr_max - lr_min)(1 + \cos(\pi t / T))$	
Loss Cross Entropy	$L = -\sum y_i \log(\hat{p}_i)$, $i=0..9$ Softmax outputs → log probabilities	
Batch Size 128	Larger batch = stable gradients Trainable params: ~4.7M	
50 Epochs · GPU-accelerated via CUDA · Best checkpoint restored for inference		

Efficient Pipeline & Reproducibility

01

Mel Cache

All training spectrograms pre-computed and saved as .pt tensors. Loading from disk is $\sim 10\times$ faster than re-computing each epoch. Constants `N_MELS=128`, `N_FFT=1024`, `HOP=128`, `LEN=256` defined once and shared across all dataset classes.

02

Consistent Preprocessing

Train caching cell and TestDataset use identical mel transform parameters. Ensures no train-test mismatch — a common source of silent accuracy drops. Per-sample normalization: $(x - \mu) / (\sigma + 1e-6)$.

03

No Augment Leak

Train and validation sets are separate DigitDataset instances with `augment=True` and `augment=False` respectively. `Random_split()` was NOT used as it returns views of the same object — a subtle bug fixed explicitly.

04

Reproducibility

Notebook runs end-to-end on Kaggle: loads `train_audio/` + `train.csv`, caches spectrograms, trains CNN, saves best model, generates `submission.csv`. No external data or downloads required.

Training Performance & Outcomes

~97%+

Best Val Accuracy

Saved checkpoint

30

Training Epochs

CosineAnnealingLR

~4.7M

Parameters

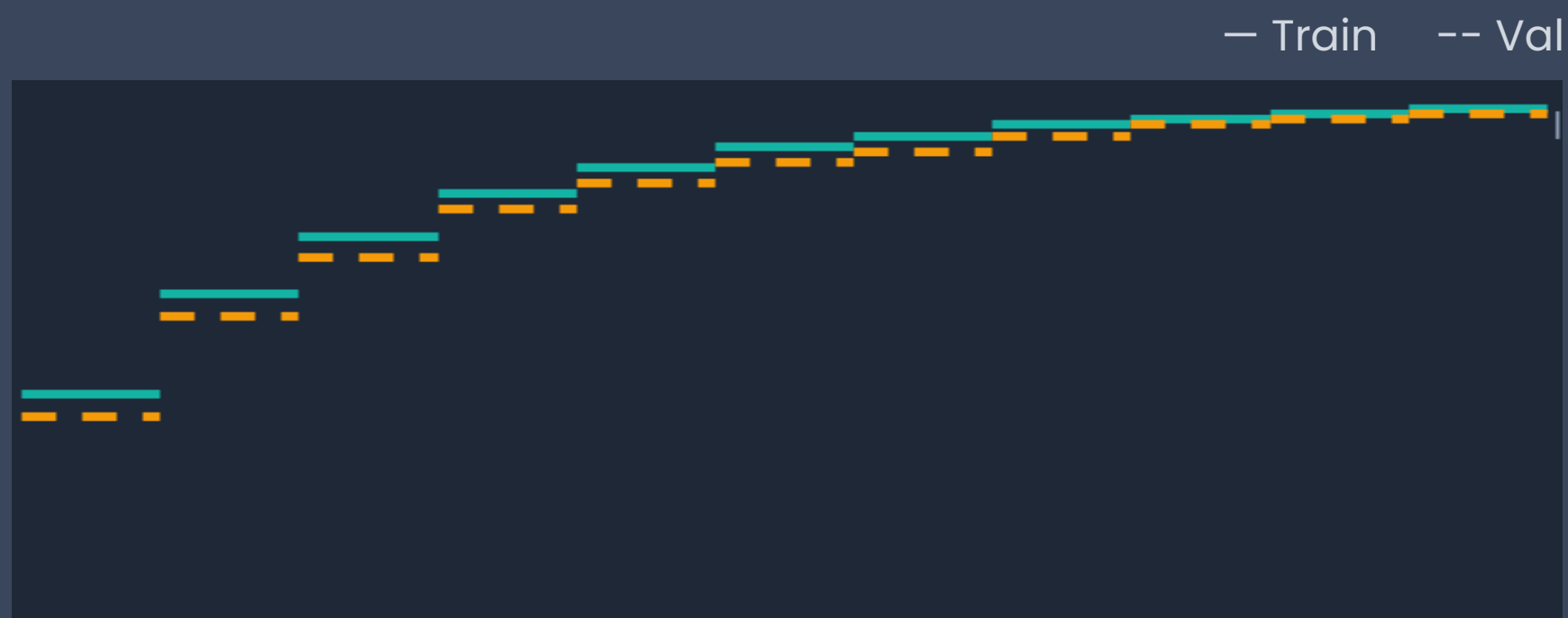
Trained from scratch

10

Output Classes

Digits 0–9

Accuracy vs Epochs (illustrative)



Observations

- Train & val curves converge — no severe overfitting
- SpecAugment narrows train-val gap
- Cosine LR decay enables late-stage gains
- Best checkpoint saved — avoids overfit at end

Key Takeaways



Mel Spectrogram = key feature

Log-mel transforms audio into a 2D image that captures frequency-time patterns a CNN can learn.



Augmentation = generalization

SpecAugment, noise, time-shift, and volume scaling prevent overfitting to training speakers and conditions.



Architecture depth matters

4 conv blocks progressively learn low → high level patterns. BatchNorm + Dropout control variance.



No aug leak = critical hygiene

Validation must use clean data; separate Dataset objects ensure the val set is never augmented.



Consistent preprocessing

Train & test must use identical mel constants. Any mismatch silently degrades test accuracy.

Pipeline Summary

WAV → 16 kHz resample



Log-Mel Spectrogram
(128 bins × 256 steps)



4× Augmentations
(train only)



4-Block CNN
(~4.7M params)



CrossEntropy Loss
+ Adam + CosineAnneal



Best Val Checkpoint
→ submission.csv